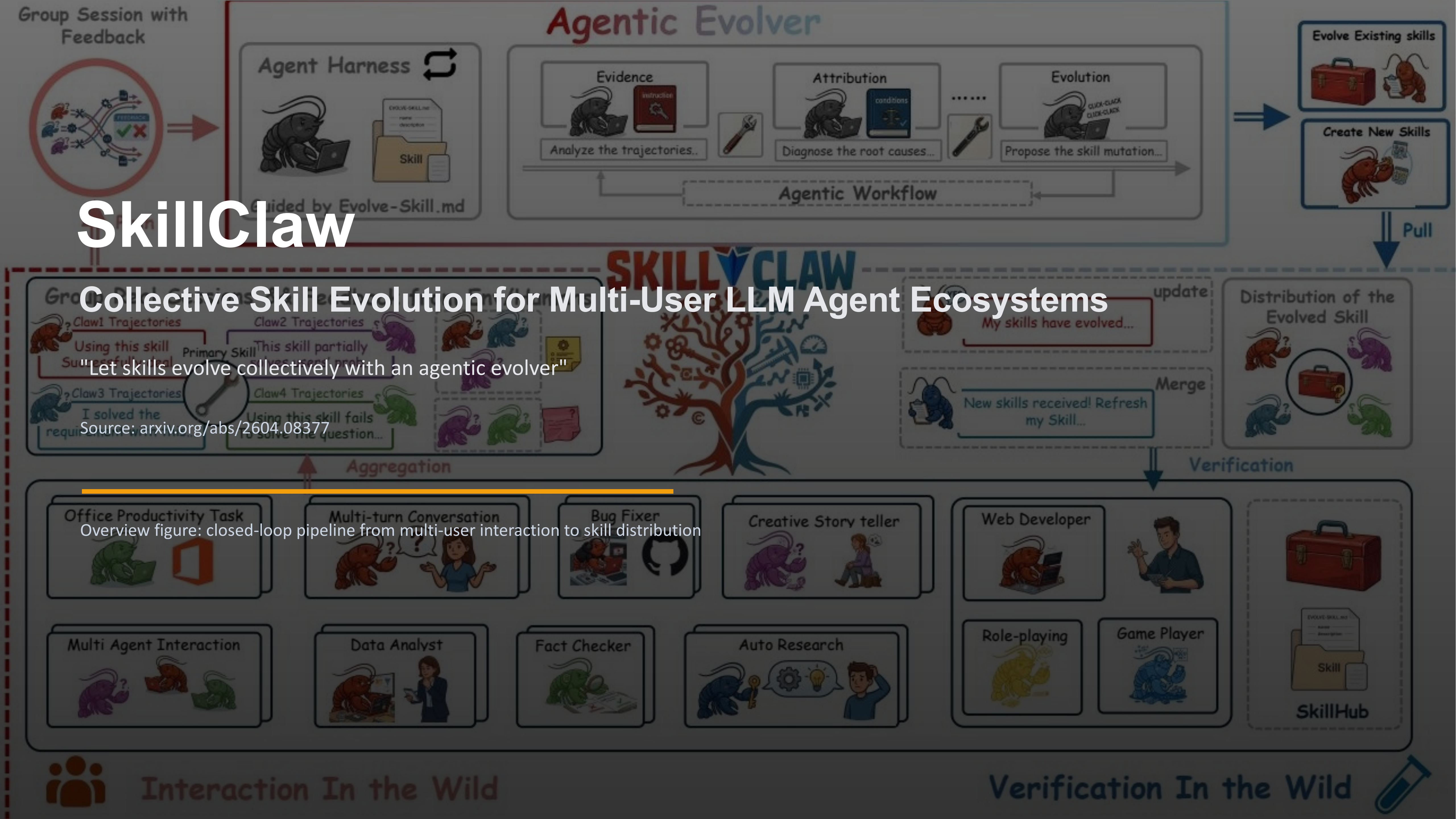
SkillClaw

Collective Skill Evolution for Multi-User LLM Agent Ecosystems

"Let skills evolve collectively with an agentic evolver"

Source: arxiv.org/abs/2604.08377

Overview figure: closed-loop pipeline from multi-user interaction to skill distribution



Problem: Static Skills After Deployment

Why multi-user agent ecosystems fail to accumulate improvements

Status quo: static skill library

- Each user rediscovers fixes
- No feedback → no updates
- Repeated failures across sessions

Desired: evolving skill ecosystem

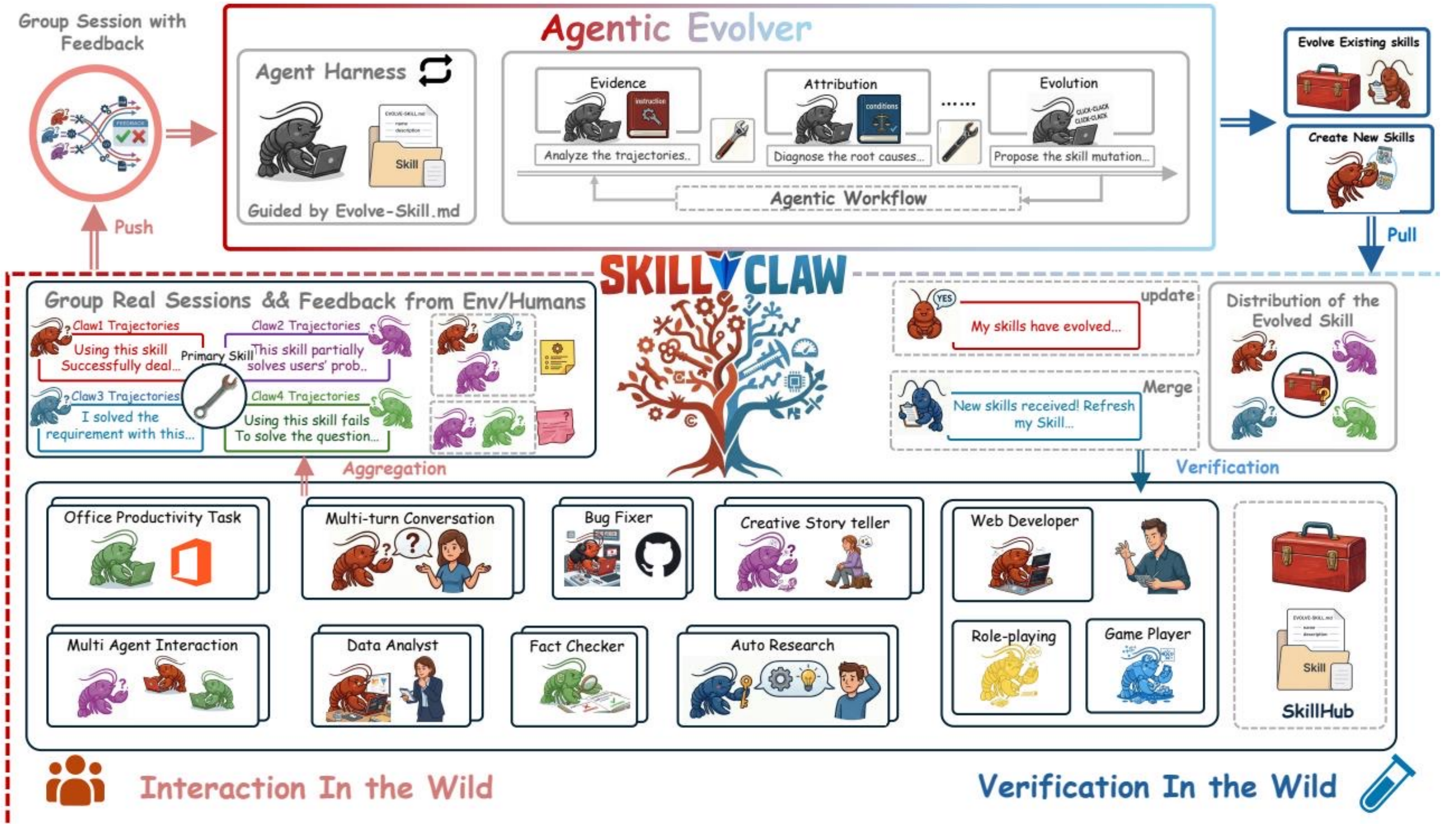
- Aggregate multi-user trajectories
- Evidence-driven skill updates
- Verified → synchronized to all agents

Key observation

- Skills are manually installed from a hub and remain static after deployment.
- Solutions discovered during interaction rarely persist beyond individual sessions.
- As tasks recur across users, failures and recoveries repeat without system-level learning.
- Memory-based methods are instance-tied; static skill libraries do not evolve through usage.

SkillClaw Architecture: Closed-Loop Collective Evolution

Evidence → Attribution → Evolution → Distribution



Four-stage closed loop

- **Evidence**
- **Attribution**
- **Evolution**
- **Distribution**

Pipeline: Multi-user Interaction → Session Collection → Skill Evolutio

Trajectories grouped by referenced skill form a shared evidence base.

Updated skills are verified before merging and synchronized back to all agents

Agentic Evolver: Evidence → Attribution → Evolution

Open-ended reasoning over interaction evidence (not predefined rules)

1) Evidence → **2) Attribution** → **3) Evolution**

1) Evidence
Analyze trajectories to identify behavioral patterns

2) Attribution
Diagnose root causes of failures and successes

3) Evolution
Propose skill mutations (refine or create new)

Verification + merge gate

- Updated skills are verified before being merged into the shared repository.
- A single autonomous agent performs the analysis and proposes updates.
- The workflow is open-ended reasoning over evidence, not hard-coded update rules.

Cross-User Aggregation Reveals Behavioral Boundaries

Complementary contexts separate generalizable improvements from one-off fixes

Aggregated evidence base

- Trajectories preserve full action–feedback causal chains
- Grouped by referenced skill across users
- Surfaces success patterns and failure modes

Compatible Claw-style systems:

OpenClaw, CoPaw, IronClaw, PicoClaw,

ZeroClaw, NanoClaw, NemoClaw

Why multi-user matters

- Diverse contexts expose where a skill works vs. where it breaks (behavioral boundar
- Single-user data is rarely sufficient to separate a generalizable fix from an idiosyncra
- Collective evidence enables conservative verification before ecosystem-wide rollout

Link back to closed loop

Multi-user interaction → session collection

provides the evidence base for evolution.

Evolver updates → synchronized skills

Case Study A: Multimodal Creative Pipeline — 6-Day Evolution Log

Conservative verification: only 1/6 candidates accepted

Each day proposes a candidate skill; validator accepts/rejects; best pool updates only on acceptance.

Source: SkillClaw creative pipeline 6-day log

Date	Outcome	Rationale
Day 1	Accepted	Check /tmp_workspace inputs & environment setup → Upgrade to new best pool
Day 2	Rejected	Multimodal input validation & output env init → Keep current best pool
Day 3	Rejected	Multimodal creative pipeline (extract from PDF/video/image) → Keep current best pool
Day 4	Rejected	Added image classification, visual generation, structured validation → Keep current best pool
Day 5	Rejected	Creative pipeline + per-file fail-fast validation → Keep current best pool
Day 6	Rejected	Extended PDF-to-poster / document-to-visual paths → Not admitted to next cycle

Accepted: 1 / 6 — conservative verification prevents regressions.

Case Study B: Git Push Auth-Fallback — 6-Day Evolution Log

Iterative refinement: 3/6 accepted before plateau

Frequent acceptances indicate evidence-supported improvements; rejections mark non-improving mutations.

Source: SkillClaw git auth-fallback 6-day log

Date	Outcome	Rationale
Day 1	Accepted	Safe fallback instead of blocking on push failure → Add to Safety best pool
Day 2	Accepted	Unified patch/bundle filenames, reduced inconsistency → Keep updated best pool
Day 3	Accepted	Auth-alternative paths & secrets audit; fixed subshell pitfalls → Keep current best pool
Day 4	Accepted	Same-pool retest; validator confirmed current pool as best → Continue same best pool
Day 5	Rejected	Added "push hang treated as auth failure"; no improvement → Keep current best pool
Day 6	Rejected	Added identity config & filename consistency; no improvement → Not admitted to next cycle

Accepted: 3 / 6 — successful refinement, then a plateau (no further improvement).

Key Takeaways & Implications

What distinguishes SkillClaw from existing systems

1) Collective evolution

Individual interactions contribute to a shared, continuously improving skill ecosystem.

2) Fully automatic

Evolution is driven by runtime interaction without manual curation or explicit user intervention.

3) Agentic evolution paradigm

Updates are produced via open-ended reasoning over evidence, not predefined update rules.

Implication: build agent ecosystems that continuously improve through shared, verified experience.